Elite Coach AI - Product Requirements Document

Audience: Frontend & Backend Engineers **Status:** Working draft **Purpose:** Tell the team what we have, what is missing, and how to build the missing pieces.

---

1. What Elite Coach AI is, in plain English

Elite Coach AI is a learning platform with three groups of users:

- **Learners** want to gain a skill or change careers. They take courses with help from an AI tutor.
- **Tutors** create lesson content. They also step in when the AI cannot help a learner.
- **Enterprise clients** are companies (banks, consulting firms) that pay us to train their staff.

We make money two ways: solo learners pay a subscription, enterprise clients pay per seat.

---

2. What we already have

Area	Status
Login + signup (learner and tutor)	Done
Goal-based learning path (one-shot, not adaptive)	Basic
Course catalog visible to learners	Done
AI sidebar for learners	Basic
One end-of-course assessment + certificate	Basic
Tutor CMS: create course, module, lesson	Done
AI ingests tutor-uploaded content (RAG)	Done

That is the whole product today. Everything else in this doc is missing.

2a. MVP Scope (what ships first)

This is the scope we are committing to for MVP. Anything not listed here is post-MVP, even if it

appears later in this doc.

IN MVP:

1. Full AI Tutor Engine: escalation logic, session management, full conversation history
2. 10 complete courses across 4 domains: Finance, Technology, Leadership & Management, Data & Analytics
3. Enterprise Admin Portal: user management, course assignment, progress dashboard, basic reporting
4. Automated certification engine: unique verification IDs, digital certs, public verify page
5. Personalized learning path (rule-based v1) with onboarding diagnostic
6. Human Tutor Inbox: escalation queue, transcript review, video response tool
7. Notification Engine: email and WhatsApp only
8. Mobile-responsive web app (no native apps yet)
9. Paystack: enterprise invoicing + individual learner payments
10. Basic analytics dashboard

NOT in MVP:

- Native iOS or Android apps
- AI Career Coach (Section A4 below is post-MVP)
- Mentorship marketplace
- ML-based path personalization
- Multi-language beyond English
- Offline mode
- Practical assignment grading (the human-graded fourth assessment type)
- SMS and in-app push notifications (email + WhatsApp only at MVP)
- AI-assist for content drafts in the CMS

Success gates for MVP (we measure these to decide if we move to full product):

Metric	Target
Enterprise clients contracted	3 (target a bank, a consulting firm, an institution)
Active learners	500 within 60 days of launch
Course completion rate	Above 65%
AI escalation rate	Below 20%
MRR	₦5M within 90 days
Critical security incidents	0
Series A data room	Populated with real platform metrics

---

3. Glossary (read this first)

- **RAG:** Retrieval-Augmented Generation. The AI looks up real lesson content first, then answers using only what it found. Stops the AI from making things up.
- **Escalation:** When the AI cannot help a learner, the session gets handed to a human tutor.
- **Skill score:** A number per skill per learner showing how strong they are at it.
- **Learner profile:** Database record holding a learner's goals, scores, weak areas, time per week.
- **RBAC:** Role-Based Access Control. Different users see different screens.
- **NDPR:** Nigeria's data protection law. We must follow it.

---

4. Roles and the panels they need

Role	Panel	Have it?
Learner	Learner Dashboard	Partly
Tutor as content author	Tutor CMS	Yes
Tutor as escalation handler	Tutor Inbox	No
Enterprise Admin	Enterprise Panel	No

Role	Panel	Have it?
Platform Admin	Admin Panel	No

A learner is either Solo (pays themselves) or Org (their company pays). Same panel, different signup flow.

SECTION A: LEARNER EXPERIENCE

A1. Onboarding and Learning Path

Today: Learner enters a goal, gets a path.

Problem: The path is built from one input. It does not test what the learner already knows. A senior developer and a beginner with the same goal get the same path.

Workflow we need:

1. Learner signs up and enters their career goal.
2. Learner answers about 10 questions in two parts: background (current role, years of experience) and a short skills check.
3. We score the skills check and save a skill profile.
4. Learner enters how many hours per week they can study.
5. We generate a path that skips skills they already have and adds reinforcement on weak ones.
6. The path becomes their main screen. The general catalog is moved behind a "Browse more courses" button.

Backend tasks:

- Tables: `skills` (with parent_id for hierarchy), `learner_profiles`, `learning_paths`, `diagnostic_questions`
- Endpoint: `POST /onboarding/diagnostic` returns a generated path
- Path generator: simple rule engine in Phase 1. "If learner has skill X at level Y, skip courses A and B."

Frontend tasks:

- Multi-step onboarding wizard (background → skills test → time per week)
- Path view as the new learner homepage
- Move catalog to a secondary route

A2. Taking a Lesson

Today: Learner reads a lesson and chats with an AI sidebar.

Problems:

- The AI sidebar may not know which lesson the learner is on.
- No knowledge checks during the lesson.
- The AI has no way to escalate to a human.

Workflow we need:

1. Learner opens a lesson.
2. The AI sidebar loads with full context: this lesson, this learner's profile, recent mistakes.
3. Short knowledge checks appear inline at the end of each section. Unscored, just to reinforce.
4. If a learner gets one wrong, the AI explains why, offers a different angle, and re-asks.
5. The learner can ask the AI anything. The AI only answers from the lesson content (RAG). It can also generate practice problems on request.
6. If the learner asks the same thing 3 times, uses frustrated language, or hits a flagged content type, the session is escalated to a tutor (see B2).
7. At the end of the lesson, the AI writes a session summary: what was covered, what was understood, what to revisit.

Backend tasks:

- Tables: `lesson_sessions`, `session_messages`, `knowledge_checks`, `escalations`
- An escalation trigger service that watches sessions and creates an `escalations` row when triggers fire
- AI prompt construction must include: lesson content from RAG, learner profile, last N messages
- Endpoint to generate and store a session summary

Frontend tasks:

- Lesson page with right-side AI panel
- Inline knowledge check components
- An "Escalated to tutor" state with clear next steps for the learner
- Session summary card at lesson end

Tip: The AI must refuse to answer if the lesson content does not cover the question, and offer to escalate instead. Otherwise tutors lose control of what learners are taught.

A3. Assessments and Certificates

Today: One assessment per course, then a certificate.

Problem: Spec needs four assessment types. Going straight to a cert after one test makes the cert weak and skips the reinforcement loop.

Four assessment types we need (long-term):

1. **Knowledge checks:** inside lessons, unscored, fast recall (covered in A2). **In MVP.**
2. **Module assessments:** scored, at the end of each module. Feed the skill profile. **In MVP.**
3. **Final exam:** at the end of the course. Pass score is configurable per course. **In MVP.**
4. **Practical assignments:** optional per course. Learner submits work, a human tutor grades it. **Post-MVP.**

If a learner fails any scored assessment, the path adds reinforcement lessons before they can retake it.

After passing the final:

- A certificate is generated with a unique verification ID (UUID).
- The Verify Certificate page lets anyone enter the ID and confirm it.
- Learner can share to LinkedIn and download the PDF.

Backend tasks:

- Tables: `assessments`, `assessment_questions`, `assessment_attempts`, `certificates`
- Endpoint: `GET /verify/:id` returns cert data or 404
- Service that creates a tutor grading task when a practical assignment is submitted
- Service that adds reinforcement lessons on assessment fail

Frontend tasks:

- Module assessment screen (different styling from final)
- Final exam screen with timer
- Practical assignment submission (file upload + text)
- Certificate page with download and LinkedIn share
- Wire Verify Certificate page to the backend (currently it does nothing)

A4. AI Career Coach

> **Post-MVP.** Not part of the launch scope. Skip this section for MVP build. Documented here so it's clear how it fits in once we move past launch.

Today: Does not exist.

A new tab in the learner panel with six features:

1. **CV review.** Learner uploads CV. AI scores it against a chosen target role and gives specific changes.
2. **Mock interview.** AI runs a 20-minute interview for the learner's target role. Returns a transcript with feedback.
3. **Career gap analysis.** Learner uploads a job description or picks one. AI compares it to learner's profile and lists what is missing.
4. **Salary benchmark.** Shows pay range for the target role in the target country.
5. **Job readiness score.** A number from 0 to 100 that updates as the learner finishes courses.
6. **Application support.** AI helps draft cover letters and recruiter outreach.

Backend tasks:

- Tables: `cv_uploads`, `mock_interviews`, `job_descriptions`, `salary_data`
- File storage (Supabase Storage or S3)
- Readiness score formula updated whenever a learner completes a lesson or assessment

Frontend tasks:

- Tabbed Career Coach section in the learner panel
- CV upload + result view
- Interview interface (text first, voice in v2)
- JD picker + gap analysis result
- Cover letter generator

Tip: Start with CV review and gap analysis. Easiest to build, highest value.

SECTION B: TUTOR EXPERIENCE

B1. Content Creation (extending the CMS we already have)

Today: Tutors create courses, modules, lessons. AI ingests them.

Gaps:

- Only text content (assumed). Need video and downloadable files. **Video + files in MVP. Embedded quizzes, case studies, interactive exercises post-MVP.**
- No tagging by skill, industry, difficulty, learning objective. **In MVP (needed for path engine to work).**
- No version history, no rollback. **Post-MVP.**
- No draft → review → publish workflow. **Post-MVP.** For MVP, content is published directly by trusted internal tutors.
- No AI-assist for first drafts. **Post-MVP.**
- No preview-as-learner mode. **In MVP.**
- No multi-language support. **Post-MVP.**
- No content analytics (which lessons cause the most escalations, which questions fail most often). **In MVP (basic version, tied to the analytics dashboard).**

Workflow we need:

1. Tutor creates a course, adds modules and lessons.
2. Each lesson supports multiple content blocks: text, video, files, embedded quiz, case study, exercise.
3. Tutor tags the lesson with skill(s), industry, difficulty, learning objective.
4. Tutor can hit "AI Draft" to generate a first version from a prompt, then edit.
5. Content saves as Draft. Tutor submits for review.
6. A Platform Admin or senior tutor approves it. Approved content gets re-indexed in the RAG store.
7. Tutor previews the course as a learner would see it.
8. Tutor sees a dashboard of lesson-level escalations and question fail rates.

Backend tasks:

- Extend `lessons` with `status` (draft, in_review, published), `version`, `language`
- New table: `lesson_versions` for history
- Tags: `content_tags` plus join tables
- Assets: `assets` table for files, video, images

- Review queue: `content_reviews` table
- Re-index service that runs on publish and updates the vector store
- Analytics: aggregations per lesson and per question

Frontend tasks:

- Lesson editor with multi-type content blocks
- Tag picker
- "AI Draft" button with prompt input
- Status badges and review submission
- Preview-as-learner mode
- Content analytics view

Tip: Treat content like code. Versioned, reviewed, and rollback-able.

B2. Tutor Inbox (Handling AI Escalations)

Today: Does not exist.

Why it matters: Without this, every AI failure is a dead end for the learner.

Workflow we need:

1. Tutor opens their Inbox and sees a queue of escalated sessions.
2. Tutor clicks one and sees the full learner-AI transcript.
3. Tutor picks one of three responses:
    - **Text reply with annotation:** quick correction shown in the learner's next session.
    - **Record a video:** short response inserted into the learner's next lesson.
    - **Schedule a 1:1:** live session booked via calendar.
4. Tutor can also send the correction back into the AI knowledge base so future learners get the right answer automatically.
5. Tutor sees their earnings. We pay per resolved case.
6. After the tutor responds, the learner is notified (in-app + WhatsApp + email) and returned to the AI flow.

Backend tasks:

- `escalations` table with state machine: open → assigned → in_progress → resolved
- Assignment logic that routes escalations to the tutor whose subject area matches
- `escalation_responses` table with type (text, video, live_session)

- Video storage
- Calendar integration (Google Calendar or Cal.com)
- `tutor_earnings` table per resolved case
- Flow for an approved correction to push into the RAG index

Frontend tasks:

- Inbox list with filters (open, in progress, urgent)
- Transcript viewer with annotation tools
- In-browser video recorder (MediaRecorder API)
- Calendar booking widget
- Earnings dashboard

Tip: This is a separate route from the CMS, not a tab inside it. Tutors will spend their work hours here.

SECTION C: ENTERPRISE ADMIN EXPERIENCE

Today: Does not exist.

Why it matters: Enterprise is the main revenue driver. Without this panel we cannot pitch Access Bank or KPMG.

C1. Setup and Team Management

1. Enterprise Admin signs up (Sales-assisted in v1).
2. Admin uploads a CSV of staff emails. System sends invites. Each invited person joins as an Org learner.
3. Admin groups staff into teams or departments.
4. Admin can deactivate users when staff leave.

C2. Assigning Courses

1. Admin picks a course or learning path.
2. Admin assigns it to one person, a team, or the whole org.
3. Admin sets a deadline.
4. Learners get notified.
5. System auto-reminds learners who fall behind.

C3. Reports and Dashboard

1. Admin sees an overview: total learners, percent completed, percent at risk, average score.
2. Admin filters by team, course, or date range.
3. Admin downloads a PDF or Excel report for compliance and board meetings.
4. Admin tracks spend per learner against their training budget.

Backend tasks:

- Tables: `organizations`, `org_memberships`, `teams`, `course_assignments`
- CSV import endpoint with row-level validation
- Report generation: PDF via Puppeteer, Excel via SheetJS
- Budget tracking fields on the organization

Frontend tasks:

- Org signup wizard
- Team management screens
- Course assignment flow
- Dashboard with charts (Recharts)
- Report export buttons

Tip: The CSV import will get bad CSVs. Build row-by-row error reporting from day one.

SECTION D: PLATFORM ADMIN EXPERIENCE

Today: Does not exist.

Why it matters: The platform team needs a UI to manage tutors, moderate content, set AI config, and handle NDPR requests without writing SQL by hand.

What it does:

- Onboard and offboard tutors
- Approve or reject submitted content
- View all escalations across the system
- Set AI config: model choice, escalation thresholds, latency targets
- Feature flags
- Payment reconciliation
- NDPR requests: data export and account deletion per user

Backend tasks:

- A `platform_admin` role in RBAC
- `admin_actions` audit log (every admin action recorded)
- `system_config` table
- Endpoints for per-user data export and cascading delete

Frontend tasks:

- A separate admin app (ideally on `admin.elitecoach.ai` subdomain)

Tip: Build the audit log on day one. If admins can change anything, you need to know who changed what.

SECTION E: CROSS-CUTTING INFRASTRUCTURE

E1. Authentication and Roles

We currently have learner and tutor. Add: `solo_learner`, `org_learner`, `tutor_author`, `tutor_responder`, `enterprise_admin`, `platform_admin`.

A user can have more than one role (some tutors author AND respond). Every API call checks role permissions. Use Supabase Auth or a Node.js JWT auth service.

E2. Notifications

Channels: email and WhatsApp at MVP. **SMS and in-app push are post-MVP.**

Minimum triggers for v1:

- Learner inactive for 3 days
- Learner failed an assessment
- Course completed (cert + share prompt)
- Assignment deadline approaching (org learners)
- Escalation resolved (notify learner)
- Escalation assigned (notify tutor)
- Weekly progress report (Monday morning)

Backend tasks:

- Tables: `notifications`, `notification_templates`
- Job queue (BullMQ or similar)
- Integrations: Resend or SendGrid for email, WhatsApp Cloud API. **SMS (Termii / Africa's Talking) and in-app web sockets are post-MVP.**

- A trigger engine that listens to events and decides what to send

Tip: WhatsApp is the most important channel for Nigerian learners. Do not deprioritize it.

E3. Payments

- Solo learner subscription (monthly + yearly) via Paystack
- Enterprise billing: manual invoicing in v1, automated in v2
- Tutor payout: per-resolved-escalation to Nigerian bank accounts via Paystack Transfers

E4. Analytics

Track every key event: lesson started, lesson completed, assessment submitted, AI message sent, escalation created, escalation resolved.

Use a single events table or a service like PostHog. Build dashboards for three audiences: internal team, enterprise admin, learner.

Key metrics: completion rate, average assessment score, AI escalation rate (target under 20 percent), time-to-completion, NPS, DAU/MAU, revenue per learner, skill score improvement.

E5. NDPR Compliance

- Privacy policy and Terms of Service (currently broken (#) links, fix this week)
- Consent collection at signup
- Per-user data export endpoint
- Per-user account deletion endpoint
- Documented data retention policy

Notes for the Team

- Do not ship a learner feature without thinking about how that data flows to the analytics dashboard and the enterprise report.
- The AI must always be grounded in published content. If it answers freely, we lose control of what learners are taught.
- Nigerian learners live on WhatsApp. Email-only notifications will not move engagement numbers.
- The four assessment types are different things in the data model. Do not flatten them into one table with a "type" column.
- Keep the tutor's two jobs (author and responder) on separate routes. They have nothing in common except the tutor's identity.

End of document. Questions and feedback to the product lead.